SecureVault

Section 7: Design Decisions

This document records our choices for the six design areas in Section 7, explains the alternatives we considered, and outlines the intended behavior. Complete byte-level formats will be specified before implementing the messages and stored objects. Measurements will be added after implementation and testing. Any unresolved design assumptions are identified explicitly.

7.1 Protecting Passwords

Choice: Argon2id (argon2-cffi library)

We chose Argon2id because its memory and computation costs make large-scale offline password guessing more expensive. PBKDF2 does not provide memory-hardness, while bcrypt has an adjustable processing cost but a small, fixed memory footprint. scrypt is also a suitable memory-hard alternative. We preferred Argon2id's configurable memory, time, and parallelism settings and its hybrid design. We will use argon2-cffi to reduce implementation workload and risk, while still checking its configuration and integration. We rejected a custom construction because it would require additional security justification and testing.

Each account will receive a fresh 16-byte salt generated by the operating system's cryptographically secure random source and stored openly with its Argon2id parameters. The salt does not need to be secret: its role is to make two identical passwords hash to different stored values and to defeat precomputed tables, not to add cost to any single guess — that cost comes from the Argon2id work factor itself, so revealing the salt gives an attacker no shortcut. We will select the memory, time, and parallelism settings after measuring performance on our machines. Approximately 0.25 seconds per verification is an initial target, not a measured result.

We will use HKDF-SHA-256 with distinct purpose labels to derive separate authentication and private-key wrapping values from the Argon2id output. The wrapping key remains client-side. The authentication value must not directly allow recovery of the wrapping key, although an attacker who successfully guesses the password can derive both. The registration and login exchanges will be specified separately.

Changing the password will require updating the authentication record and re-encrypting the existing private keys under a newly derived wrapping key. These updates must remain consistent if the operation is interrupted.

> ***Scenario.*** *Layla registers with a fresh salt. If the credential database is stolen, an attacker must perform the configured Argon2id computation for each password guess. Key separation prevents direct use of the stored authentication value to decrypt her private keys, but does not eliminate password guessing.*

7.2 Protecting Document Contents

Choice: ChaCha20-Poly1305, implemented by our team

AES-GCM would require implementing AES and GHASH, while AES-CCM would require AES with CTR encryption and CBC-MAC authentication. We preferred ChaCha20-Poly1305 because ChaCha20's addition, rotation, and XOR operations offer a manageable implementation approach. Poly1305 still requires careful arithmetic, formatting, and testing; Python's big integers do not automatically provide constant-time execution.

We will follow the standardized authenticated-encryption construction, which authenticates ciphertext and associated data, rather than create our own composition.

We follow the Encrypt-then-MAC approach defined by ChaCha20-Poly1305: authentication covers the ciphertext and associated data, and the client rejects an invalid tag before releasing plaintext.

Each new document encryption will use a fresh 256-bit document key and a fresh 96-bit nonce, both generated using the operating system's cryptographically secure random source. Each document key will be used for only one encryption operation. The nonce will be stored openly with the encrypted document. ChaCha20-Poly1305 internally derives the one-time Poly1305 authentication key from that key and nonce; we do not generate a separate independent MAC key for document encryption. Keys used to protect the document key for recipients have a separate role and will be derived through ECDH and HKDF.

A retry may resend an already completed encrypted object unchanged. If encryption must run again after a failure, it will use fresh key material rather than reuse an earlier key and nonce. Secure random generation makes accidental key repetition negligibly unlikely; it does not make it mathematically impossible.

Filename, type, size, timestamp, owner, and document ID will be authenticated as associated data. They remain visible to the server, but unauthorized changes are detected. The client must also check that the authenticated document ID matches the object it requested.

> ***Scenario.*** *Layla uploads a file with a fresh document key; if an attacker flips a ciphertext byte or edits the filename, authentication fails and Omar's client rejects the object outright.*

7.3 Public-Key Cryptography

Choice: ECDH over P-256 for key agreement; RSA-PSS (3072-bit) for signatures

We selected P-256 for ECDH and RSA-3072 for RSA-PSS, targeting approximately 128-bit classical security. ECDH gives elliptic-curve cryptography a direct role in sharing and uses smaller public values than finite-field Diffie–Hellman at comparable security. RSA key transport is another valid option, but ECDH fits our planned shared-secret derivation approach. Without public-key mechanisms, users would need another way to establish shared secrets, and shared-key authentication alone would not provide publicly verifiable signatures.

P-256 has standardized parameters and conventional point arithmetic. X25519 is a valid alternative at a similar security level, while P-384 exceeds our selected target. We will measure relevant operation times and report the comparison with finite-field methods rather than infer speed from key sizes alone.

For signatures, ECDSA and DSA require careful handling of a secret per-message value, which may be generated randomly or through a standardized deterministic method. Reusing that value across different messages can expose the private key. Reusing an RSA-PSS salt does not cause the same direct key-recovery failure, although we will still generate fresh random salts. We accept RSA's larger keys and signatures for this tradeoff.

RSA-2048 provides approximately 112-bit security, below our target, so we selected RSA-3072. RSA-PSS will use SHA-256, MGF1-SHA-256, and a 32-byte random salt with strict verification. ChaCha20 retains its required 256-bit key, but the public-key parameters limit our intended classical security level to approximately 128 bits.

> ***Scenario.*** *Layla's client runs ECDH with Omar's P-256 key to protect the document key, then signs the encrypted object with her RSA-3072 key; Omar verifies it without ever touching the plaintext.*

7.4 Trusting a Public Key

Choice: Trust-on-first-use (TOFU) with optional out-of-band fingerprint verification

On first contact, the client records the other user's public-key fingerprint and allows interaction without requiring an external comparison. This assumes that the first public-key record received is genuine. The client pins that record and blocks later operations if either public key changes, rather than silently accepting the replacement.

The SHA-256 fingerprint covers a canonical encoding of the user ID, P-256 public key, and RSA public key. This allows changes to either key to be detected. The client distinguishes a record trusted through TOFU from one independently verified by the user.

Users may compare fingerprints through a trusted external channel to strengthen the identity binding. The comparison must establish both the other person's identity and the matching fingerprint. A channel is not trustworthy merely because it is outside SecureVault. A changed key requires independent verification before replacing the pinned record.

We chose TOFU because it supports first contact without operating a certificate authority or requiring users to arrange an external verification step before sharing. Mandatory fingerprint comparison provides stronger first-contact authentication but adds a required user step. A certificate authority requires a trusted signing key and identity-validation procedures. Verification codes also require an authenticated comparison method.

The main limitation is that TOFU does not authenticate the first key. An attacker who substitutes a key during the initial exchange may be trusted and remain undetected. Optional fingerprint verification helps only when users complete it correctly; users who skip it remain exposed to this attack. Our key-continuity guarantee therefore depends on an authentic first contact.

> ***Scenario:*** Layla's client records Omar's fingerprint when they first interact. If that initial record is genuine, a later substitution is detected and sharing is blocked. If the attacker substitutes the first record, TOFU alone cannot detect it.

7.5 Signatures Versus MACs

We use MACs and signatures for different purposes. Poly1305 provides authentication within document encryption, while HMAC-SHA-256 is used within HKDF. HMAC-based login challenge–response and dummy derivation for unknown usernames are planned uses whose complete behavior still needs specification and testing; their presence alone does not establish secure login or prevent timing leaks.

RSA-PSS will authenticate the encrypted document and the relevant sharing context. The signed representation will bind the ciphertext and tag to the metadata, document and user identifiers, key fingerprints, freshness values, ephemeral public key, and wrapped document key. The exact field order and encoding will be defined consistently so the bytes being verified are unambiguous.

A digital signature supports third-party verification because verification requires only the public key, while a MAC's shared secret allows either holder to generate valid tags. However, a signature proves authorship by a named user only when the signing key is reliably associated with that user.

Our TOFU approach preserves key continuity after an assumed authentic first contact. Optional verification through a trusted external channel strengthens that association. If the initial key was substituted and never independently verified, a valid signature does not establish the claimed person's identity. Our authorship and non-repudiation claims are therefore conditional on the authenticity of the trusted public-key record.

Incorrect comparison or compromise of the trusted external channel can still undermine this guarantee.

7.6 Architecture, Formats and Verification

The client and server will communicate through files in **a shared directory** that simulates the network. We chose this approach over sockets to reduce networking work and focus on cryptographic behavior and testing. We retain a client–server structure rather than peer-to-peer communication because it fits centralized storage and asynchronous sharing. The shared directory will be treated as attacker-controlled: its contents may be read, modified, or replayed.

The server will store accounts, public keys, encrypted documents, wrapped private keys, and share records. Document encryption and private-key recovery will take place on the client, so plaintext document keys and unwrapped private keys will not be placed in the shared directory or sent to the server.

What the server is structurally prevented from doing

These are capabilities the server lacks because it never holds the material a given attack would need, not behaviors we merely trust it to avoid. It cannot read a document's contents, because the document key reaches it only wrapped under keys derived from ECDH secrets whose private half it never sees. It cannot open a private key, because the wrapped key object is encrypted under a value derived from the user's password, which never reaches the server. It cannot forge authorship, because signing requires a private key the server never holds. It cannot alter metadata undetected, because metadata is authenticated as associated data under the document key and, for shared objects, is also covered by the sender's signature. It cannot substitute a public key undetected once a user has seen the real one, because our trust-on-first-use fingerprint would no longer match; this protection does not cover the first exchange. It cannot replay an object undetected, because freshness state is kept on the recipient's own machine, not on the server. The one place the server's position gives it a real opening is that first exchange, which is the limitation already stated under Decision 7.4.

Preliminary object formats

We plan to begin every object with version and type fields, use 4-byte length prefixes for variable-length fields, and encode integers in big-endian order. The table below summarizes the planned object contents. The complete format specification will define field widths, text encoding, limits, and protocol messages.

Object	Main fields	Size / format
Credential	Version, type, user ID, salt, Argon2 parameters, authentication value	Salt: 16 B Auth value: 32 B
Public key	Version, type, user ID, P-256 public key, RSA modulus, RSA public exponent	P-256: 65 B RSA modulus: 384 B
Wrapped private key	Version, type, user ID, nonce, ciphertext, tag	Nonce: 12 B Tag: 16 B
Document	Version, type, document ID, owner, filename, MIME type, size, timestamp, nonce, ciphertext, tag	Nonce: 12 B Tag: 16 B
Share	Version, type, document ID, sender, recipient, key fingerprints, sequence number, timestamp, ephemeral key, wrapped key, signature	Ephemeral P-256 public point: 65 B Signature: 384 B

Every signed object's serialization is deterministic, so the same logical object always produces the same bytes to sign or verify.

Verification and failure handling

On receiving a shared object, the client will validate its format, check the sender's key against the applicable trust rules, verify the signature, and apply freshness checks. It will then recover the document key and verify the document's AEAD tag before releasing plaintext. A valid signature establishes authorship only to the extent that the signing key has been reliably associated with the sender.

Failed objects will be rejected without updating trusted-key or replay-tracking state. The client will report a limited set of errors, such as Malformed object, Key mismatch, Verification failed, or Stale object.

How the properties fit together: The signature binds the encrypted document to its sender and sharing context. The AEAD tag authenticates the ciphertext and associated metadata under the document key. Signed freshness fields cannot be altered undetected, but rejecting replay also requires acceptance rules and retained state; a timestamp or sequence number alone is insufficient. The exact freshness policy will be specified before implementation.